

Problem Solving using Python (CSE1012)

Presented By:

Dr. Neeraj Kumar Misra

Associate Professor
Department of SENSE,
VIT-AP University

Google Scholar: https://scholar.google.co.in/citations?user=_V5Af5kAAAAJ&hl=en

Research Gate profile: https://www.researchgate.net/profile/Neeraj_Kumar_Misra

ORCHID ID: <https://orcid.org/0000-0002-7907-0276>



Function in Python

➤ Function is a block of code which run only when you call for it.

Types of function

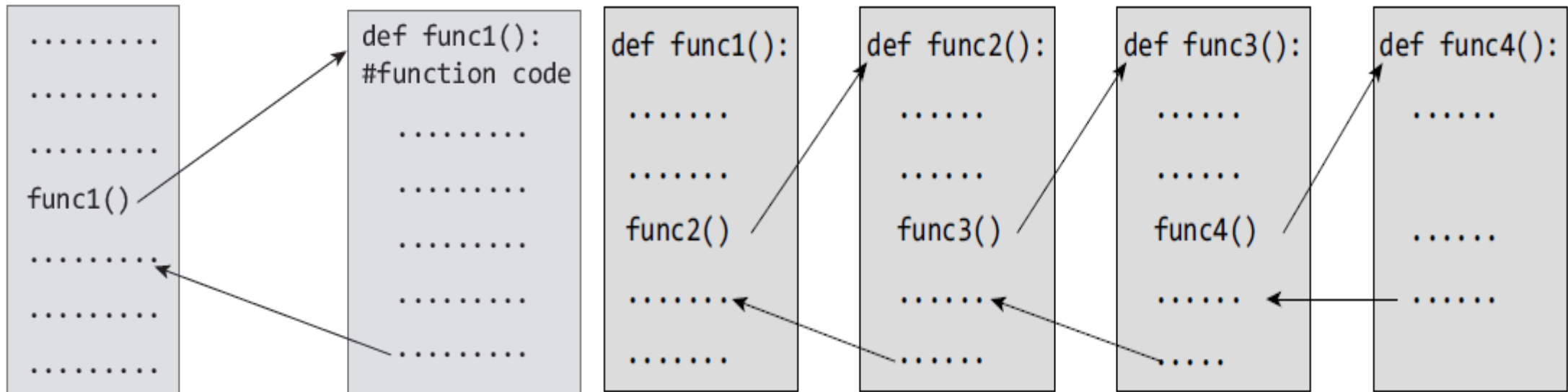
1. Build in function: print(), upper() etc
2. User defined function: User to build this function

General syntax of function

```
def function_name():  
    local variable  
    block of statement  
    return (variable)
```

Functions

Python enables its programmers to break up a program into segments commonly known as functions, each of which can be written more or less independently of the others. Every function in the program is supposed to perform a well-defined task.



```
def diff(x,y):          # function to subtract two numbers
    return x-y
a = 20
b = 10
operation = diff        # function name assigned to a variable
print(operation(a,b))   # function called using variable name
```

OUTPUT

10

```
def func(i):  
    print("Hello World", i)  
func(5+2*3)
```

OUTPUT

Hello World 11

```
def func():  
    for i in range(4):  
        print("Hello World")  
func()      #function call
```

OUTPUT

Hello World
Hello World
Hello World
Hello World

```
def total(a,b):    # function accepting parameters  
    result = a+b  
    print("Sum of ", a, " and ", b, " = ", result)
```

```
a = int(input("Enter the first number : "))  
b = int(input("Enter the second number : "))  
total(a,b) #function call with two arguments
```

OUTPUT

Enter the first number : 10
Enter the second number : 20
Sum of 10 and 20 = 30

```
def func(i):          # function definition header accepts a variable with name i  
    print("Hello World", i)  
    j = 10  
    func(j)           # Function is called using variable j
```

OUTPUT

Hello World 10

Calculated area of a circle with a radius of 5 units.

Useful steps

1. **import math**
2. **use def and take one parameter**
3. **Inside take apply formula formula**
4. **area=pi*radius**2**
5. **area= $\pi \times \text{radius}^2$**
6. **return value**
7. **define radius value**
8. **calculate are using function**
9. **print: radius and area**

User-defined function

```
# Calculated area of a circle with a radius of 5 units.  
import math  
  
def circle_area(radius):  
    pi = math.pi  
    area = pi * radius**2  
    return area  
  
radius = 5  
  
area = circle_area(radius)  
  
print(f"The area of the circle with radius {radius} is: {area:.2f}")
```

How to call a function

```
def add():
```

```
    x = 10
```

```
    y = 5
```

```
    z = x + y
```

```
    print(z)
```

```
add()
```

How to call a function

```
def add(y):
```

```
    x=5
```

```
    z=x+y
```

```
    print(z)
```

```
    add (6)
```

e.g.1

```
def trail_function():  
    print("VIT-AP University")  
trail_function()
```

e.g.2

```
def trail_function(x):  
    print(x + " University")  
trail_function("VIT-AP")
```

e.g.3

```
def trail_function(x, y):  
    print(x + " " + y)  
trail_function("VIT-AP", "University")
```

Math function

E.g.1

```
import math  
print(math.pow(3,2))
```

E.g.2

```
import math  
print(math.sqrt(16))
```

E.g.3

```
import math  
x=[2,3,7.2]  
print(math.fsum(x))
```

E.g.4

```
import math  
x=5  
print(math.factorial(x))
```

To find the Square of the number

Useful Steps

1. **Define function name (def square()): Donot pass any parameters**
2. **take input from user**
3. **use formula $x**2$**
4. **return variable**
5. **call by remote function square()**
6. **print**

square of the number

```
def square():  
    x = int(input("Enter the number: "))  
    y = x**2  
    return y  
n = square()  
print(n)
```

Local and Global Variables

Example

```
num1 = 10    # global variable
print("Global variable num1 = ", num1)
def func(num2):    # num2 is function parameter
    print("In Function - Local Variable num2 = ", num2)
    num3 = 30    # num3 is a local variable
    print("In Function - Local Variable num3 = ", num3)
func(20)    # 20 is passed as an argument to the function
print("num1 again = ", num1)    # global variable is being accessed
#Error- local variable can't be used outside the function in which it is defined
print("num3 outside function = ", num3)
```

OUTPUT

```
Global variable num1 = 10
In Function - Local Variable num2 = 20
In Function - Local Variable num3 = 30
num1 again = 10
num3 outside function =
Traceback (most recent call last):
  File "C:\Python34\Try.py", line 12, in <module>
    print("num3 outside function = ", num3)
NameError: name 'num3' is not defined
```

Programming Tip: Variables can only be used after the point of their declaration

Use of Global and local variable

```
var = "Good"
def show():
    global var1
    var1 = "Morning"
    print("In Function var is - ", var)
show()
print("Outside function, var1 is - ", var1)    #accessible as it is global
variable
print("var is - ", var)
```

OUTPUT

```
In Function var is -  Good
Outside function, var1 is -  Morning
var is -  Good
```

Programming Tip: All variables have the scope of the block.

Lambda Functions Or Anonymous Functions

- Any number of **input parameters** but can only have one expression
- We are **not using def** keywords and return the expression but not values
- Apply any number of **arguments**
- Do not use **global variables**
- Use **'lambda'** as keywords
- Often used for short, and simple operations

```
lambda arguments: expression
```

```
sum = lambda x, y: x + y  
print("Sum = ", sum(3, 5))
```

OUTPUT

```
Sum = 8
```

1. Write a python program to multiply two numbers
2. Write a python program to square of the list

Useful hints

- Use lambda and pass argument then define expression
- Print all operations

Example 1:

```
multiply = lambda x, y: x * y  
print("Result of multiplication:", multiply(3, 4))
```

Example 2:

Using lambda function with built-in functions

```
numbers = [1, 2, 3, 4, 5]  
squared_numbers = list(map(lambda x: x**2, numbers))  
print("Squared numbers:", squared_numbers)
```

Squared numbers: [1, 4, 9, 16, 25]

Write a python program to filtered all even number from the list

Useful hints

- Use lambda and pass argument then define expression
- Print all operations

```
# Filtering even numbers from a list
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print("Even numbers:", even_numbers)
```

Write a python program to Calculating the power of a number using a lambda function

Useful hints

- Use lambda and pass argument then define expression
- Print all operations

```
power = lambda base, exponent: base ** exponent  
result = power(2, 3)  
print("2^3 =", result)
```

Write a python program to find addition, subtract, multiply, divide using lambda function

Useful hints

- Use lambda and pass argument then define expression
- Print all operations

```
# Dictionary with lambda functions for basic operations
calculator = {
    'add': lambda x, y: x + y,
    'subtract': lambda x, y: x - y,
    'multiply': lambda x, y: x * y,
    'divide': lambda x, y: x / y if y != 0 else "Cannot divide by zero"
}

# Using the dictionary
print("Addition:", calculator['add'](5, 3))
print("Substraction:", calculator['subtract'](5, 3))
print("Multiplication:", calculator['multiply'](4, 6))
print("divide:", calculator['divide'](4, 2))
```

Recursive Functions

- To call itself and repeat its behaviours to meet results
- Function called itself.
- The recursion occurs when the function calls itself with a smaller version of the problem (a smaller list in this case) until it reaches the base case.

Practice set Questions

1. Write a Python program to reverse the string using recursive function
2. Write a Python program to find factorial using recursive function

Calculate Factorial

Useful Steps

- Defines a recursive function named factorial that takes an integer `n` as input.
- `if n == 0 or n == 1::` This line checks if `n` is equal to 0 or 1. If true, it returns 1 because the factorial of 0 and 1 is 1.
- If `n` is greater than 1, the function returns the product of `n` and the result of calling itself with `n-1`. This is the essence of the recursive calculation of the factorial.
- calls the factorial function with the argument 5 and stores the result in the variable `result`
- Print factorial

To find factorial of the given number

```
def fact(n):  
    if(n==1 or n==0):  
        return 1  
    else:  
        return n*fact(n-1)  
n = int(input("Enter the value of n : "))  
print("The factorial of",n,"is",fact(n))
```

OUTPUT

```
Enter the value of n : 6  
The factorial of 6 is 720
```

To reverse the string

Useful Steps

1. Defines a recursive function named `reverse_string` that takes a string (`str1`) as input.
2. checks if the length of the input string is 1. If it is, the function returns the string itself, as a string of length 1 is its own reverse.
3. If the length of the string is greater than 1, the function calls itself recursively with the substring starting from the second character (`str1[1:]`) and appends the first character at the end (`str1[0]`).
4. This process continues until the base case (string length is 1) is reached.
5. calls the `reverse_string` function with the user-input string and stores the reversed string in the variable `str2`
6. Prints the original input string and its reversed version.

Reverse the string using function

```
def reverse_string(str1):  
    if len(str1) == 1:  
        return str1  
    else:  
        return reverse_string(str1[1:]) + str1[0]  
  
str1 = input("Enter the string: ")  
str2 = reverse_string(str1)  
print(f"The reversed string is: {str2}")
```

Write a Python program to find the sum in the list

Useful Steps

- The function takes a list `lst` as its parameter.
- The base case checks if the list is empty (`not lst`). If it is, the function returns 0, indicating that the sum of an empty list is 0.
- If the list is not empty, the function returns the sum of the first element of the list (`lst[0]`) and the result of the `sum_list` function called with the rest of the list (`lst[1:]`).
- Finally, the result is printed

```
def sum_list(lst):  
    if not lst:  
        return 0  
    else:  
        return lst[0] + sum_list(lst[1:])
```

```
# Example usage: pass a list to the function  
my_list = [1, 2, 3, 4, 5]  
result = sum_list(my_list)  
print(result)
```

Pause

amul

```
def reverse_str(str1):  
    if len(str1) == 1:  
        return str1  
    else:  
        return reverse_str(str1[1:])+str1[0]
```

```
str1 = input("enter the string:")  
str2 = reverse_str(str1)  
print(str2)
```

maul

```
def reverse_str(str1):  
    if len(str1) == 1:  
        return str1  
    else:  
        return reverse_str(str1[1:])+str1[0]
```

lu

l

```
def reverse_str(str1):  
    if len(str1) == 1:  
        return str1  
    else:  
        return reverse_str(str1[1:])+str1[0]
```

u

```
def reverse_str(str1):  
    if len(str1) == 1:  
        return str1  
    else:  
        return reverse_str(str1[1:])+str1[0]
```

l + u

Formal and Actual Parameters

```
def avg(x1, x2, x3):  
    return (x1+x2+x3) / 3  
x=avg(2, 3, 4)  
y=avg(5, 6, 7)  
z=avg(8, 9, 10)  
print(x)  
print(y)  
print(z)
```

Formal and Actual Parameters

```
def multiply_numbers(a, b):  
    result = a * b  
    print(f"The product is: {result}")  
  
multiply_numbers(a=5, b=3)  
multiply_numbers(b=4, a=2)
```

To check the given number ($n=10$) is even or odd

Useful steps

1. defines a function called `is_even` that takes a parameter number
2. checks if the remainder of number divided by 2 is equal to 0. If true, the function returns True; otherwise, it returns False
3. assigns the value 10 to the variable num
4. checks if the function `is_even` returns True for the value of num
5. If True, it prints that the number is even; otherwise, it prints that the number is odd.

return statement based Python Program

```
# Checking if a Number is Even
def is_even(number):
    return number % 2 == 0
num = 10
if is_even(num):
    print(f"{num} is even.")
else:
    print(f"{num} is odd.")
```

Types of Arguments

1. Required arguments
2. Keywords arguments
3. Default arguments
4. Variable arguments

Required arguments

- Number of argument should be same in both function call and function definition order
- Position of argument should be followed

• eg.

```
def display (a,b):
```

```
    print(a,b) → 20, 5
```

```
display (a=20, b=5) → Function call (2 argument passed)
```

Function definition
(2 argument passed)

Keywords arguments

- Order or position is not required
- Initialization will be done based on keywords

• Eg.

```
def display (a,b)
```

```
    print(a,b) → 5, 20
```

```
display (b=20, a=5)
```

Note:
Arguments: Input given to the function

Default arguments

- Number of arguments need not be match with both function call and function definition
- Sum of argument will be considered as default argument

- eg.

```
def display (name, university='KLU'):
    print(name)
    print(university)

display (name='emerging university')
display(name='vit')
```

```
emerging university
KLU
vit
KLU
```

Variable length arguments

- Accept any number of arguments
- By placing * as prefix to the argument of function definition
- eg.

```
def display(*T20_world_cup_2023):
    for i in T20_world_cup_2023:
        print(i)

display("India", "Australia", "England")
```

```
India
Australia
England
```

Function in Python

Group of related statements that performs a specific task.

As the program grows larger and larger, functions make it more organized and manageable.

Syntax :

```
def function_name(parameters):  
    """docstring"""  
    statement(s)
```

NOTE: Add docstring to define what a function does.

Click [here](#) to learn about **built-in python function**

The diagram illustrates the syntax of a Python function definition with the example: `def celsius_to_fahr(temp):
 return 9/5 * temp + 32`. Labels with arrows point to specific parts of the code: 'def keyword' points to 'def', 'name' points to 'celsius_to_fahr', 'parameter' points to 'temp', 'return statement' points to 'return', and 'return value' points to the expression '9/5 * temp + 32'.

```
def keyword      name      parameter  
  ↓              ↓          ↓  
def celsius_to_fahr(temp):  
  return 9/5 * temp + 32  
  ↑           ↖  
return statement  return value
```

Write a function called primes that is given a number n and return a list of the first n primes let the default value of n be 100

Useful steps

1. Define primes function generates a list of prime numbers up to a specified count (n)
2. while loop to continue generating primes until the list (l) has the desired number of elements.
3. for loop inside the while loop checks for divisibility of x by numbers from 2 to the square root of x. If x is divisible by any of these numbers, it breaks out of the loop. If the loop completes without finding a divisor, x is a prime number, and it is added to the list.
4. Takes user input to determine how many prime numbers should be generated.
5. Generate and Print Prime Numbers

```
def primes(n=100):  
    l = []  
    x = 2  
    while len(l) < n:  
        for i in range(2, int(x**0.5) + 1):  
            if x % i == 0:  
                break  
        else:  
            l.append(x)  
            x += 1  
    return l  
  
num_primes = int(input("Enter the number of primes required: "))  
result = primes(num_primes)  
print("List of the first", num_primes, "prime numbers:", result)  
print("List of the first", n, "prime numbers:", primes(n))
```

Looping and Data structures in Functions

Looping: Repetition of a set of instructions until a specific condition is met.

- for loops,
- while loops,
- do-while loops

Data structures in Functions



```
list1=[1,2,3]
list2=[4,5,6]
list1.extend(list2)
list1
```

```
list1=[1,2,3]
list2=[3,4,5]
list1.insert(1,6)
list2.insert(0,7)
print(list1)
print(list2)
```

```
[1, 6, 2, 3]
[7, 3, 4, 5]
```

```
list1=[1,1,2,3,4,5]
x=list1.count(1)
y=list1.count(3)
print(x)
print(y)
```

2
1

```
list1 = [1, 1, 2, 3, 4, 5]
x = list1.pop(3)
print(x)
```

3

Note: pop element remove the element at index 3 (which is the fourth element in the list)

Find the output of the following Python code



```
list5 = ['apple', 'banana', 'cherry', 'date']  
fruit = list5.pop(1)  
print(fruit)
```

banana

```
list4 = [1, 2, 3, 4, 5, 6]  
value_to_remove = 4  
w = list4.remove(value_to_remove)  
print(w)
```

None

```
list6 = [7, 14, 21, 28, 35]  
element = list6.pop(2)  
list6[1] = list6[1] * 2  
print(element, list6)
```

21 [7, 28, 28, 35]

```
list3 = [100, 200, 300, 400, 500]  
z = list3.pop()  
print(z)
```

500

Find the output of the following Python code



```
list1 = [10, 20, 30, 40, 50]
removed_element = list1.pop()
print("Removed Element:", removed_element)
print("Updated List:", list1)
```

```
Removed Element: 50
Updated List: [10, 20, 30, 40]
```

```
list2 = [100, 200, 300, 400, 500]
index_to_remove = 2
removed_element = list2.pop(index_to_remove)
print("Removed Element:", removed_element)
print("Updated List:", list2)
```

```
Removed Element: 300
Updated List: [100, 200, 400, 500]
```

```
list = [1, 2, 3, 2, 4, 2, 5]
value_to_remove = 2
while value_to_remove in list:
    list.remove(value_to_remove)
print("Updated List:", list)
```

```
Updated List: [1, 3, 4, 5]
```

```
list = [7, 14, 21, 28, 35]
list.clear()
print("Updated List:", list)
```

```
Updated List: []
```

```
list = ['apple', 'banana', 'cherry', 'date']
index_to_remove = 1
removed_element = list[index_to_remove]
del list[index_to_remove]
print("Removed Element:", removed_element)
print("Updated List:", list)
```

```
Removed Element: banana
```

```
Updated List: ['apple', 'cherry', 'date']
```

Ackerman Function

- The Ackermann function is a mathematical function that is notable for its extremely rapid growth. It is often used as a benchmark for testing the effectiveness of various algorithms, particularly those involving recursion

$$A(m,n) = \begin{cases} n + 1 & \text{if } m=0 \\ A(m-1,1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m-1, A(m,n-1)) & \text{if } m > 0 \text{ and } n > 0 \end{cases}$$

where m and n are non-negative integers

Examples of Ackerman Function

➤ E.g.1 $A(0,5)=6$ >>m=0

E.g. 2 $A(1,2)=A(0,A(1,1))=A(0,2)=3$

$$\begin{aligned} A(1,1) &= A(1,0) \\ &= A(0,1) \\ &= 2 \end{aligned}$$

Ackerman Function

```
def ackermann(m, n):  
    if m == 0:  
        return n + 1  
    elif m > 0 and n == 0:  
        return ackermann(m - 1, 1)  
    else:  
        return ackermann(m - 1, ackermann(m, n - 1))  
  
# Example usage:  
result = ackermann(3, 4)  
print(result)
```

Practise Questions based on Data structures in Functions

Write a python program to count the number of key value pair in the given dictionary. Ask the user to input a key k and delete the key k from the dictionary. If the key is not present in the dictionary display the message "Key not Found"

Sample input and output

```
Number of key-value pairs in the dictionary: 5
Enter a key to delete: b
Key 'b' deleted successfully.
Updated dictionary: {'a': 1, 'c': 3, 'd': 4, 'e': 5}
```

Useful hints

- ✓ Define a Sample Dictionary.....>>>Use { }
- ✓ Count the Number of Key-Value Pairs....>>> Use len
- ✓ Ask User for Input:
- ✓ Check if the Key Exists:.....>>>Use if
- ✓ Delete the Key if it Exists:.....>>>Use del statement
- ✓ Handle Key Not Found Case:.....>>> Use else condition
- ✓ Display the Modified Dictionary:.....>>>Use print

```
my_dict = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
```

```
num_pairs = len(my_dict)
```

```
print(f"Number of key-value pairs in the dictionary: {num_pairs}")
```

```
key_to_delete = input("Enter a key to delete: ")
```

```
if key_to_delete in my_dict:
```

```
    del my_dict[key_to_delete]
```

```
    print(f"Key '{key_to_delete}' deleted successfully.")
```

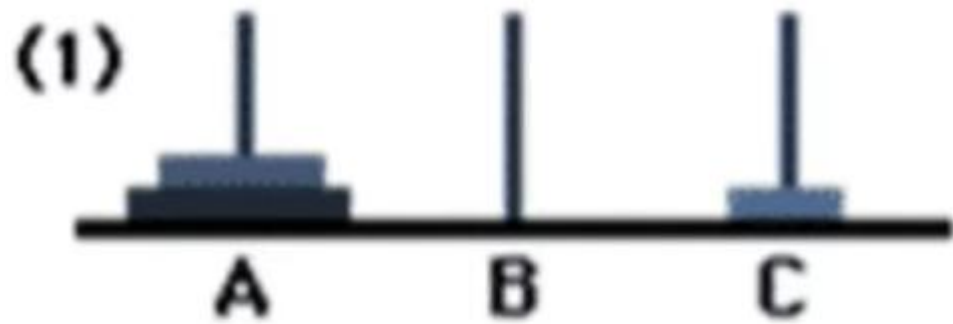
```
else:
```

```
    print("Key not found in the dictionary.")
```

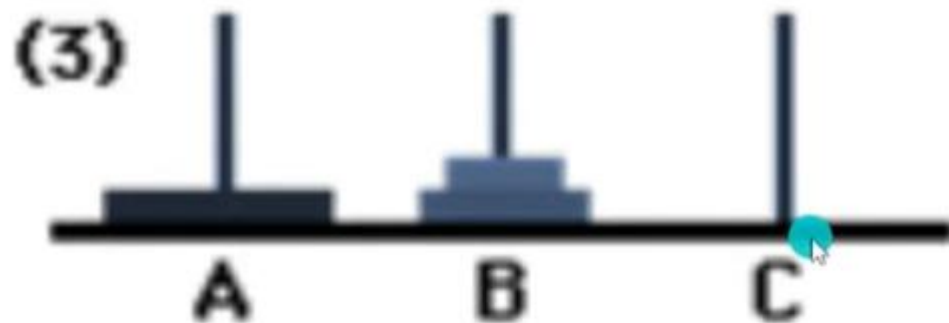
```
print("Updated dictionary:", my_dict)
```

Tower of Hanoi in Data Structures and Algorithms

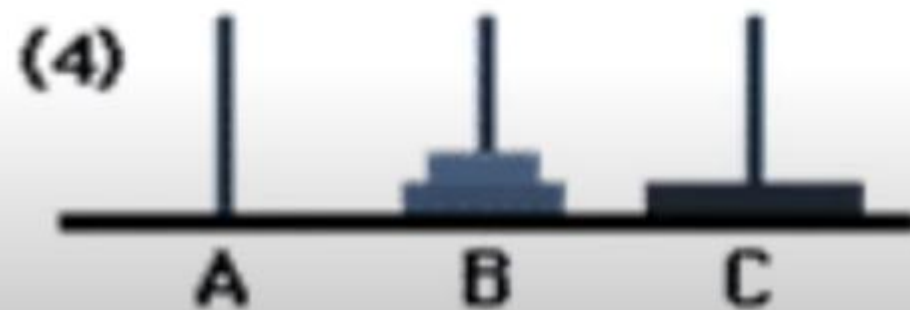
It is a mathematical puzzle game in which we use three identical rods and n disks, all of the different sizes. The discs are arranged in the first rod, such that the largest disc is placed at the bottom and the smallest at the top. To solve the puzzle, one needs to arrange the disc in the same order in the last rod via the middle rod.



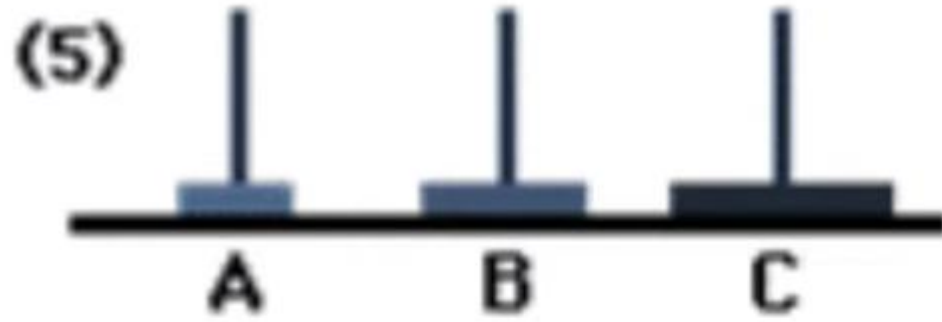
Now, we will move the disc from A to B



After this, we will move the disc from A to C



Now, move the disc from B to A



Next, we need to move the disc of rod B to C



And for the last step, we will move the disc from A to C, thus solving the puzzle



Tower of Hanoi in Data Structures and Algorithms

Tower of Hanoi: This is the mathematical puzzle when we have three rods (A,B,C) and N disk

- Here, we have used recursive method for the implementation of the game
- The function TowerOfHanoi() takes four parameters
 - Number of discs
 - Source rod
 - Auxiliary rod
 - Destination rod
- It first checks the condition if the number of disc is 1, it directly moves it to the Destination rod and terminates the function
- Then, we have recursively called the function to move the remaining n-1 discs from source node to the auxiliary node, using the destination rod as the auxiliary
- After that, the 1 disk that is remaining is directly moved from the source rod to the auxiliary rod
- Lastly, we have again recursively called the the function to move the remaining n-1 rods from the auxiliary rod to the destination rod, using the source node as auxiliary

Tower of Hanoi in Data Structures and Algorithms

```
def TOH(number, start, aux, end):  
    if number == 1:  
        print("Move disk 1 from rod{} to rod{}".format(start, end))  
        return  
    TOH(number-1, start, end, aux)  
    print("Move disk {} from rod{} to rod{}".format(number, start, end))  
  
    TOH(number-1, aux, start, end)  
    disk=3  
# Example usage:  
TOH(disk, 'A', 'B', 'C')
```

Why use Functions in Python: Clarity

Replace multiple lines of code with a single function call, thereby making it easier to understand and maintain

Without Function

```
print('Hello World')
print('Hello World')
print('Hello World')
print('Hello World')
print('Hello World')
print('Hello World')
print('Hello World')
print('Hello World')
print('Hello World')
```

With Loop

```
count = 10
while count>0:
    print("Hello World")
    count-=1
```

With Loop with Function

```
def print_msg(count, msg):
    while count>0:
        print(msg)
        count-=1

#Calling the defined function
print_msg(10, 'Hello World')
```

Why use Functions in Python: Reuse

Tried and tested functions can be reused multiple time within/outside the program.

```
print_msg(10, 'Hello World')  
print_msg(15, 'Hola')
```

Why use Functions in Python: Division of Labour

Several programmers can divide the workload in a large project by working on different functions for the program

CODER 1

```
def add(n1,n2):  
    return n1+n2
```

CODER 2

```
def sub(n1,n2):  
    return n1-n2
```

CODER 3

```
def mul(n1,n2):  
    return n1*n2
```

CODER 4

```
def div(n1,n2):  
    return n1/n2
```

NOTE: Above is just an example which symbolizes that all the functions (sub task) can be worked upon in parallel among the team. These functions can later be combined together in a main function.

Function in Python: Example

Let's say you need to build a calculator to find the factorial of any number given by the user.



Logic to
implement
Factorial

```
def fact(n):  
    factorial = 1  
    if int(n) >= 1:  
        for i in range (1,int(n)+1):  
            factorial = factorial * i  
    return factorial
```

NOTE: Here '*fact*' is **USER DEFINED FUNCTION** and '*n*' is a **function argument**

Function Definition

fact (4)

Factorial of 4

fact (6)

Factorial of 6

fact (8)

Factorial of 8

NOTE: Here '*fact*(4)', '*fact*(6)', '*fact*(8)' are examples of **function calling**

Function Call

24

720

40320

Output

Function Argument



The number of arguments in the function call should match exactly with the function definition

Example:

```
def print_name(str):  
    print("Welcome to Python, ",str)  
    return()  
  
print_name()
```

Result:

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-11-aa4d22772bb0> in <module>  
      3     return()  
      4  
----> 5 print_name()  
  
TypeError: print_name() missing 1 required positional argument: 'str'
```

Arguments are missing in the function call. Hence, it returned an error.

Scope of a Variable - Functions

Global Variables



SCOPE

The variables which are declared outside the function and can be used anywhere in the program

```
a = 50
def number():
    b = 30
    print(b)

print(a)
number()
```

Local Variables

The variables which are declared within any function can be used within that function only

```
a = 50
def number():
    b = 30
    print(b)

print(a)
number()
```

Scope of a Variable - Functions

```
a = 30
def sum(b):
    c=30
    sum = b+c
    print('Addition is', sum)
sum(3)
print('Value of c', c)
```

Variable 'a' is a global variable. It can be accessed anywhere in the program

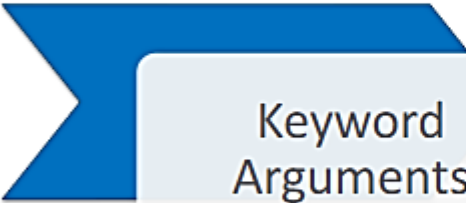
Result:

```
Addition is 33
-----
NameError                                Traceback (most recent call last)
<ipython-input-53-a10954081147> in <module>()
      5 print('Addition is', sum)
      6 sum(3)
----> 7 print('Value of c', c)

NameError: name 'c' is not defined
```

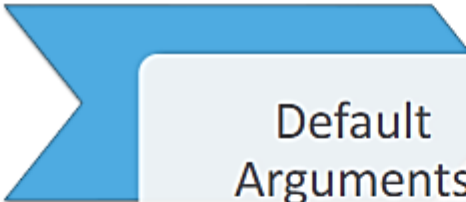
Variable 'c' is a Local variable. So, it can't be accessed outside the function

Types Of Function Arguments



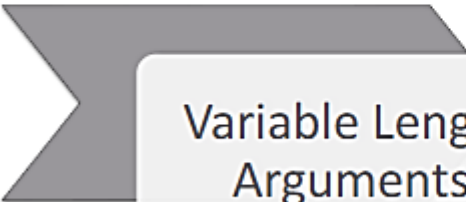
Keyword
Arguments

Keyword argument is an argument, provided with a variable name while passing the values in the function call



Default
Arguments

Default argument chooses a default value if a value is not provided in the function call



Variable Length
Arguments

When you need to process a function for more arguments than the specified arguments defined in the function, *variable length arguments* can be used. Also known as *Arbitrary arguments*

Example - Keyword Arguments

Arguments position doesn't matter when you give the argument names in the function call

Problem:

```
def substring1(start_index, stop_index, str1):  
    print("Substring is: ", str1[start_index:stop_index])
```

```
substring1("Example of Keyword Argument", 10, 30)
```

Returns Error

Solution:

```
def substring1(start_index, stop_index, str1):  
    print("Substring is: ", str1[start_index:stop_index])
```

```
substring1(str1="Example of Keyword Argument",  
start_index=10, stop_index=30)
```

Example - Default Arguments

Let the function take the default values when you miss to give the value of an argument in the function call

Problem:

```
def info(name, age):  
    print("Name: ", name)  
    print("Age :", age)  
    return()  
info(age=25, name="John")  
info(name="Michael")
```

```
Name: John  
Age : 25
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-8-172ee5b76626> in <module>  
      4     return()  
      5     info(age=25, name="John")  
----> 6     info(name="Michael")  
TypeError: info() missing 1 required positional argument: 'age'
```

Solution:

```
def info(name, age=50):  
    print("Name: ", name)  
    print("Age :", age)  
    return()  
info(age=25, name="John")  
info(name="Michael")
```

```
Name: John  
Age : 25  
Name: Michael  
Age : 50
```

Example - Variable Length Arguments

A single variable or multiple variables can be passed as an argument to the same function

Problem:

```
def purchaseinfo(product1, product2):  
    print("Products purchased:")  
    print(product1, product2)  
    return()
```

```
purchaseinfo("Chocolates", "Milk")  
purchaseinfo("Cookies")
```

```
Products purchased:  
Chocolates Milk
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-21-4b02b348407c> in <module>  
      5  
      6 purchaseinfo("Chocolates", "Milk")  
----> 7 purchaseinfo("Cookies")
```

```
TypeError: purchaseinfo() missing 1 required positional argument: 'product2'
```

Solution:

```
def purchaseinfo(product1, *products):  
    print("Products purchased:")  
    print(product1)  
    for item in products:  
        print(item)  
    return()
```

```
purchaseinfo("Chocolates", "Milk")  
purchaseinfo("Cookies")
```

```
Products purchased:  
Chocolates  
Milk  
Products purchased:  
Cookies  
  
()
```